Inside `mandelHybrid`: An Architecture and Code Guide

Matias Saavedra

Friday 10th July, 2026

Binary: `main` at tag `binary-v5-affinity` (`fc33e29`) — 9-point stencil + GPU-affinity min-max work queue, the current best version. This is a code/architecture reference, not a measurement campaign; quantitative results are attributed to the reports that produced them (01–13).

Resumen

This report is an end-to-end guide to the `mandelHybrid` source: what each file does, how the pieces fit, and why the design balances load the way it does. The program renders a 100-frame Mandelbrot zoom by recursively splitting each frame into rectangular regions and computing them on a pool of CPU threads plus a single GPU thread, all pulling from one shared work queue. The load-balancing strategy is adaptive subdivision: a region whose sampled iteration counts are uniform (or which is already small) is computed directly; otherwise it splits into four and the children go back on the queue. The queue is executor-aware — the GPU thread takes the largest pending region, CPU threads the smallest — which routes the big coherent interior regions onto the processor that is ∼30× faster on them. We walk every translation unit (`main.cpp`, `mandelframe`, `mandelregion`, `workqueue`, `kernel.cu`) with real code excerpts, explain the Qt and CUDA idioms a C++ reader may not know, and close with how the design's measured behaviour (∼49.9 s wall, GPU ∼91 % utilised, a conserved ∼595 s of compute) follows from the code. Companion file: `INSTRUMENTATION.md`.

1. The Big Picture

`mandelHybrid` turns a camera path through the complex plane into a sequence of PNG images. The unit of work is a region: a rectangle of pixels belonging to one frame. The whole program is a producer/consumer loop over regions (Figure 1).

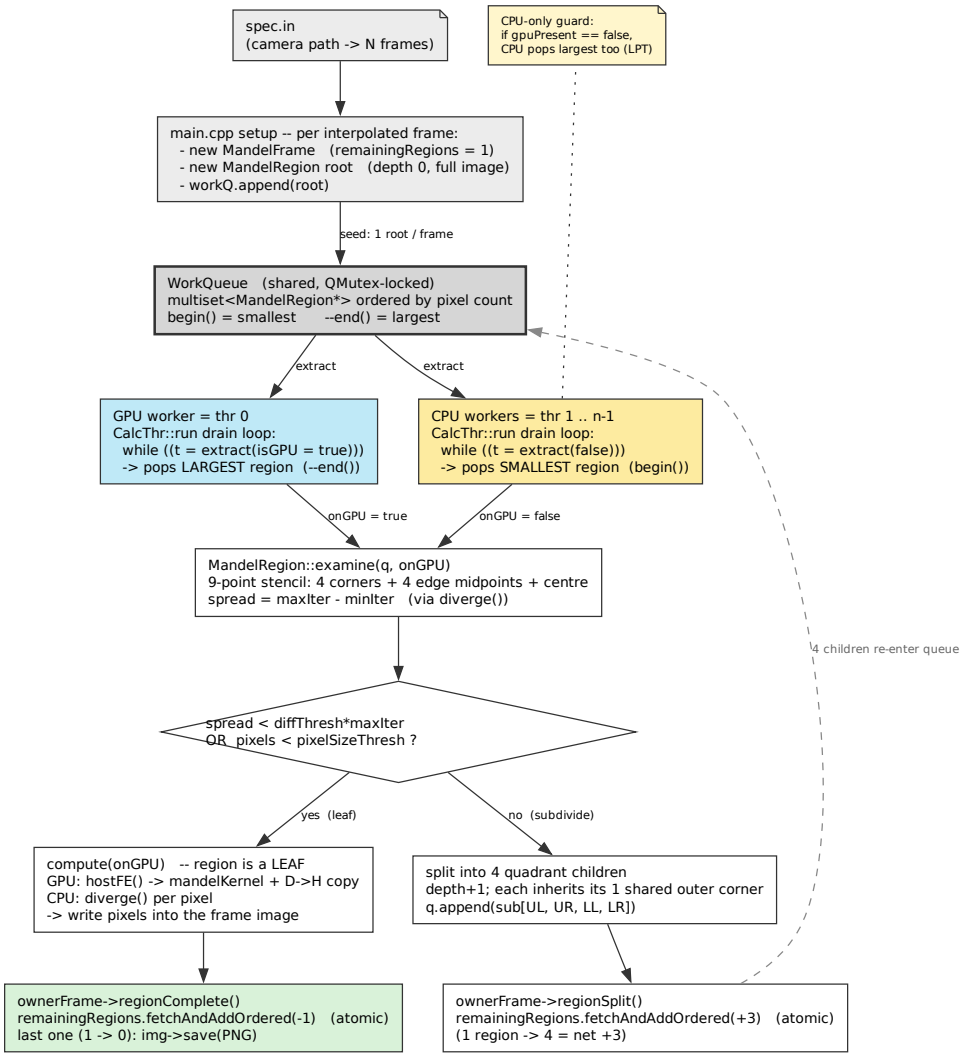

Figura 1: The whole program as a producer/consumer cycle over regions, drawn directly from the source at `fc33e29`. Read top to bottom: `main.cpp` seeds the shared `WorkQueue` with one root `MandelRegion` per interpolated frame, and every worker thread runs the same `CalcThr::run` drain loop. The load-balancing decision lives in two places the figure highlights. Who pulls a region: cyan is the single GPU thread, which pops the largest pending region (`--end()`); yellow is the CPU pool, which pops the smallest (`begin()`); the dotted note is the no-GPU LPT guard. How big the task is: `examine`'s 9-point spread test. From there a region is either a leaf (`compute`, then the atomic `regionComplete` whose last $1 \to 0$ decrement writes the PNG) or a split (four quadrant children re-enter the queue along the dashed edge, `regionSplit` bumping the same atomic counter by +3). Those three local decisions — task owner, task size, completion — are the entire scheduler.

Execution model.

One `CalcThr` (a `QThread` subclass) runs per logical worker. Thread 0 drives the GPU path; every other thread is CPU-only. All of them run the same drain loop: pull a region, examine it, repeat until the queue is empty. "Examine" is the load-balancing decision — it either computes the region's pixels or replaces the region with four smaller ones. A region therefore lives somewhere in a quadtree: the frame is the root, and the leaves are the rectangles actually computed.

What "adaptive subdivision" buys.

The Mandelbrot set is cheap to evaluate where points escape quickly (a handful of iterations) and expensive deep inside the set (every pixel runs to `MAXITER`). A fixed pixel-per-task split would put wildly different costs in equal-sized tasks. Subdivision instead makes task size track visual complexity: smooth regions stay large and are computed in one shot; busy boundary regions are split until each piece is either uniform or tiny. Figure 2 shows the result — the quadtree is fine along the fractal boundary and coarse in the flat interior and exterior, and each leaf is coloured by the processor that computed it.

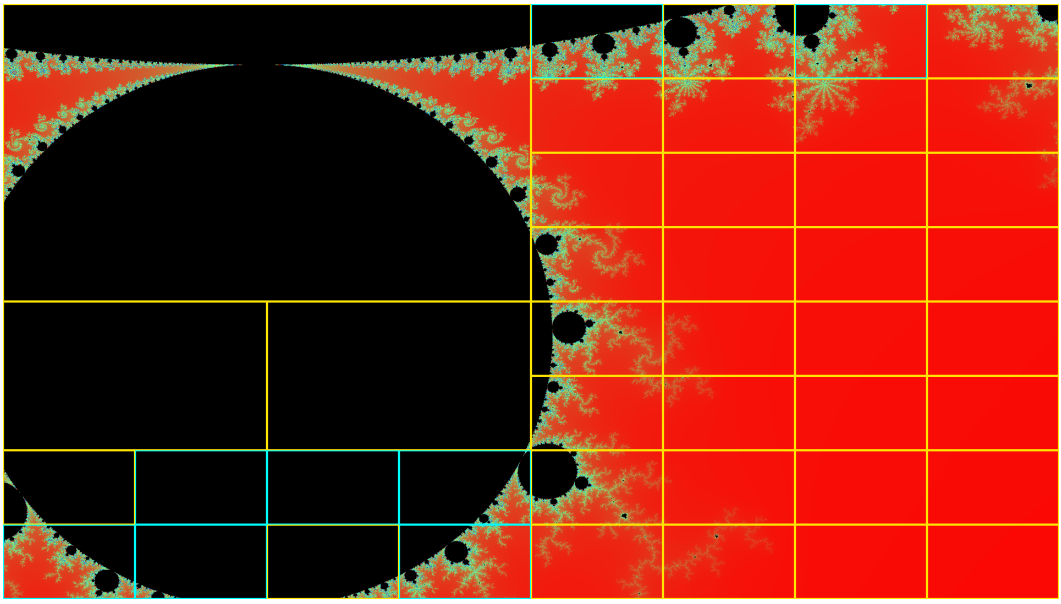

Figura 2: Frame 89's terminal partition, coloured by executor: cyan leaves were computed on the GPU thread, yellow on a CPU thread, the grey skeleton marks internal (split) nodes. Note the dense subdivision tracing the minibrot boundary and the few large cyan rectangles in the coherent interior — exactly the big regions GPU affinity routes to the GPU. Rendered with `viz=1` (see $\mathcal{S}$8.1).

Files at a glance.

Five translation units, flat at the repository root (matching the Makefile):

Tabla 1: The source tree. "LB role" is each file's part in load balancing.

File	Responsibility	LB role
`main.cpp`	arg parsing, setup, threads, wall timer	launches workers
`mandelframe.*`	per-frame image + atomic completion counter	detects "frame done"
`mandelregion.*`	the subdivision decision + CPU compute	the heuristic
`workqueue.*`	thread-safe min-max region queue	the dispatch policy
`kernel.cu/.h`	CUDA kernel + host front-end	the GPU compute path

The next sections take them in dependency order: the orchestrator, the frame, the region (the heart), the queue, and the GPU kernel.

2. main.cpp — Orchestration

`main` parses the command line, builds the frames and their root regions, launches the worker threads, times the compute phase, and (optionally) emits the visualization. It owns no algorithmic logic — it wires everything together.

Configuration flags shared across files.

Three integers are defined here with C linkage and referenced (via `extern "C"`) from the other translation units, so one command-line switch controls behaviour everywhere. C linkage keeps the symbol name identical across the `nvcc` and `g++` object files.

Código 1: main.cpp — the cross-file flags.

```
extern "C" int profileQuiet = 0;   // 1 = suppress per-region stderr prints
extern "C" int profileSave  = 1;   // 0 = skip PNG saves (pure-compute timing)
extern "C" int vizMode      = 0;   // 1/2/3 = subdivision-animation modes
```

Argument parsing.

Positional, all optional after the spec file, each guarded by `argc`. `numThreads` defaults to the online CPU count; `viz` ≥ 2 forces `save` $= 0$ because the overlaid PNGs are the deliverable.

Código 2: main.cpp — positional argument parsing (excerpt).

```
int numThreads = sysconf(_SC_NPROCESSORS_ONLN);
if (argc > 2) numThreads = atoi(argv[2]);
bool enableGPU = true;
if (argc > 3) enableGPU = (bool) atoi(argv[3]);
if (argc > 4) diffT      = atof(argv[4]);
if (argc > 5) pixT       = atoi(argv[5]);
if (argc > 6) profileQuiet = atoi(argv[6]);
if (argc > 7) profileSave  = atoi(argv[7]);
if (argc > 8) vizMode      = atoi(argv[8]);
if (argc > 9) vizFrame     = atoi(argv[9]);
if (vizMode >= 2) profileSave = 0;   // overlaid PNGs are the deliverable
```

Frames and root regions.

The spec gives the first and last camera rectangle plus a max-iteration count for each; `main` interpolates `numframes` steps linearly between them. Each frame becomes a `MandelFrame`, and each frame gets exactly one root `MandelRegion` covering the whole image, pushed onto the queue. That single root is what later fans out into the quadtree.

Código 3: main.cpp — building the interpolated frames (viz=0/2/3 path).

```
for (int i = 0; i < numframes; i++) {
    sprintf(fname, "%s%04i.png", imageFilePrefix, i);
    fr[i] = new MandelFrame(uX, uY, lX, lY, resolutionX, resolutionY, fname, iter);
    fr[i]->frameIndex = i;
    workQ.append(new MandelRegion(uX, uY, lX, lY, 0, 0,
                                  resolutionX, resolutionY, fr[i]));
    uX += sx1;  uY += sy1;          // advance the camera corners
    lX += sx2;  lY += sy2;
    iter += iterInc;               // and the iteration budget
}
```

Note the steps `sx1`..`sy2` are the per-frame increments; the comment in the source records a subtlety — they are divided by `numframes` (not `numframes-1`) so accumulated round-off cannot make the last frame fall short of its target rectangle.

Threads.

`numThreads` workers are created. Thread 0 is special: it is the GPU worker (when `enableGPU`), and `main` runs it on the main thread itself rather than spawning it, so the main thread is not idle. Threads $1 \ldots n-1$ are CPU workers, started with `QThread::start()`.

Código 4: main.cpp — worker launch and the wall timer.

```
CalcThr **thr = new CalcThr*[numThreads];
thr[0] = new CalcThr(&workQ, enableGPU);        // thread 0 = GPU worker
for (int i = 1; i < numThreads; i++) {
    thr[i] = new CalcThr(&workQ, false);        // the rest = CPU workers
    thr[i]->start();
}

struct timespec t0, t1;
clock_gettime(CLOCK_MONOTONIC, &t0);            // <-- start of measured
    ↪ phase
if (enableGPU) {
    CUDAmemSetup(resolutionX, resolutionY);     // allocate device + pinned host
    thr[0]->run();                              // run GPU worker inline
    CUDAmemCleanup();                           // prints the GPU summary
} else
    thr[0]->run();
for (int i = 1; i < numThreads; i++) thr[i]->wait();  // join CPU workers
clock_gettime(CLOCK_MONOTONIC, &t1);            // <-- end of measured phase
double elapsed_s = (t1.tv_sec - t0.tv_sec) + (t1.tv_nsec - t0.tv_nsec) * 1e-9;
fprintf(stderr, "[total_elapsed_s] %.6f\n", elapsed_s);
```

The timer brackets the entire compute phase, including the PNG saves (they happen on whichever CPU worker finishes a frame last — see $\mathcal{S}3$) and the GPU summary print. `CLOCK_MONOTONIC` is immune to NTP/DST jumps. The visualization passes run after `t1`, so they never count against the measurement.

The worker loop.

`CalcThr::run()` is the consumer. Each thread drains the queue, examining and deleting regions until `extract` returns `NULL`, then prints its CPU timing summary (a no-op on the GPU thread).

Código 5: main.cpp — the universal worker drain loop.

```
void CalcThr::run() {
   MandelRegion *t;
   while ((t = que->extract(isGPU)) != NULL) {
      t->examine(*que, isGPU);     // decide: compute, or split + re-enqueue
      delete t;                    // this region object is consumed
   }
   MandelRegion::printCPUSummary(); // per-thread CPU stats (GPU thread:
    ↪ nothing)
}
```

The same loop runs on every thread; the only difference is the `isGPU` flag threaded through to `extract` (which end of the queue to pull) and to `examine`/`compute` (which compute path to take).

3. mandelframe — the Image and the Completion Counter

A `MandelFrame` owns one output image and the bookkeeping that decides when that image is finished. Because a frame's regions are computed concurrently by many threads, "finished" cannot be a simple boolean — it is an atomic counter.

Código 6: mandelframe.h — the frame and the VizRect record.

```
struct VizRect {            // one recorded subdivision node (viz only)
  int  x, y, w, h;          // pixel rectangle within the frame
  int  depth;               // recursion depth (root = 0)
  bool leaf;                // true if computed, false if split into four
  bool onGPU;               // executor (meaningful only when leaf)
};

class MandelFrame {
public:
   int MAXITER;
   int frameIndex;                // position in the interpolated sequence
   double upperX, upperY, lowerX, lowerY;
   double stepX, stepY;           // complex units per pixel
   int pixelsX, pixelsY;
   QImage *img;
   char fname[MAXFNAME+1];
   QAtomicInt remainingRegions;   // atomic frame-completion counter
```

```
    QVector<VizRect> vizRects;      // populated only when vizMode != 0
    QMutex vizLock;
    void regionSplit();
    void regionComplete();
    void addVizRect(const VizRect &r);
};
```

The completion invariant.

`remainingRegions` starts at 1 (the root). The trick is in how splits and completions adjust it. When a region splits, one region becomes four — a net increase of three pending regions. When a region is computed, one pending region disappears. The frame is done exactly when the counter reaches zero.

Código 7: mandelframe.cpp — the atomic counter and the deferred save.

```
void MandelFrame::regionSplit() {
 remainingRegions.fetchAndAddOrdered(3);   // 1 region -> 4 = net +3
}

void MandelFrame::regionComplete() {
 if (remainingRegions.fetchAndAddOrdered(-1) == 1) {  // was 1, now 0: last one
   if (profileSave)
     img->save(fname, "PNG");           // the finishing thread saves the PNG
 }
}
```

`fetchAndAddOrdered` is Qt's atomic read-modify-write: it returns the value before the add and carries a full memory barrier ("Ordered"), so the thread that drives the counter from 1 to 0 is the unique one that saves, with no torn reads of the pixel buffer. This is why the save time lands inside the wall timer on an arbitrary CPU worker, and why no separate "join the frame" step is needed — completion detects itself. `addVizRect` guards `vizRects` with `vizLock` because every worker appends to it concurrently; it is only ever called on the `vizMode` path, so the lock never touches a timed run.

4. mandelregion — the Algorithmic Core

This is where the load-balancing heuristic and the CPU pixel math live. Three methods matter: `diverge` (the Mandelbrot iteration), `examine` (the split-or-compute decision), and `compute` (turn a region into pixels).

4.1. diverge: the escape-time iteration

The kernel of the whole program. For a complex point c, iterate $z \leftarrow z^2 + c$ from $z = c$ until $|z|^2 \geq 4$ (escape radius 2) or the iteration budget runs out. The return value is the escape iteration — low near the exterior, exactly `MAXITER` for points in (or numerically indistinguishable from) the set.

Código 8: mandelregion.cpp — the escape-time iteration (CPU copy).

```
int MandelRegion::diverge(double cx, double cy) {
  int MAXITER = ownerFrame->MAXITER;
  int iter = 0;
  double vx = cx, vy = cy, tx, ty;
  while (iter < MAXITER && (vx * vx + vy * vy) < 4) {
    tx = vx * vx - vy * vy + cx;    // Re(z^2 + c)
    ty = 2 * vx * vy + cy;          // Im(z^2 + c)
    vx = tx;  vy = ty;
    iter++;
  }
  return iter;
}
```

Everything costly about this workload is here: in-set points run the full MAXITER loop (up to 10,000 at the deepest zoom), so a region full of in-set pixels is the most expensive thing the program can be handed. An identical `___device___` copy lives in `kernel.cu` so the CPU and GPU produce byte-identical images.

4.2. examine: the split-or-compute decision

`examine` is the load balancer. It samples the region, decides whether the region is "uniform enough" or small enough to compute as-is, and otherwise splits it into four quadrants that go back on the queue.

Step 1 — sample the four corners.

Corners are cached (`cornersIter[]`), because a child inherits one corner from its parent (the shared outer corner), so it is never recomputed.

Código 9: mandelregion.cpp — corner evaluation with the bug-fixed reduction.

```
int minIter = INT_MAX, maxIter = 0;
for (int i = 0; i < 4; i++) {
   if (cornersIter[i] == UNKNOWN) {
      switch (i) {                          // each corner = one diverge()
        case UPPER_RIGHT: cornersIter[i] = diverge(lowerX, upperY); break;
        case UPPER_LEFT:  cornersIter[i] = diverge(upperX, upperY); break;
        case LOWER_RIGHT: cornersIter[i] = diverge(lowerX, lowerY); break;
        default:          cornersIter[i] = diverge(upperX, lowerY);  // LOWER_LEFT
      }
   }
   if (cornersIter[i] < minIter) minIter = cornersIter[i];   // independent
   if (cornersIter[i] > maxIter) maxIter = cornersIter[i];   // min/max tracking
}
```

The two independent `if`s are the one-line bug fix of report 03. The original code chained them as `if (min) ... else if (max)`, which made the `max` update unreachable whenever the same sample also lowered the min — most visibly on `i==0`, where `minIter` starts at `INT_MAX` so the min branch always fires and the first corner could never set `maxIter`. About 25 % of regions had their true maximum silently discarded, some then misclassified as uniform and not split. The fix exposed ∼9–

12 % more leaves (report 04).

Step 2 — the 9-point stencil.

Four corners are blind to a high-iteration filament that threads between them. The stencil adds the four edge midpoints and the centre. The five interior samples are always fresh (a child never inherits a midpoint), so each `examine` costs five extra `diverge` calls over the 4-corner version — measured negligible (report 09).

Código 10: mandelregion.cpp — the 9-point stencil (corners + 4 edge midpoints + centre).

```
double midX = (upperX + lowerX) * 0.5;
double midY = (upperY + lowerY) * 0.5;
int extra[5] = {
    diverge(midX,   upperY),   // top edge midpoint
    diverge(midX,   lowerY),   // bottom edge midpoint
    diverge(upperX, midY),     // left edge midpoint
    diverge(lowerX, midY),     // right edge midpoint
    diverge(midX,   midY)      // centre
};
for (int i = 0; i < 5; i++) {
    if (extra[i] < minIter) minIter = extra[i];
    if (extra[i] > maxIter) maxIter = extra[i];
}
```

Step 3 — the decision and the split.

The rule is: compute if the sampled iteration spread is small relative to the maximum, or the region is already below the pixel floor; otherwise split.

Código 11: mandelregion.cpp — the uniformity rule and the four-way split.

```
if (maxIter - minIter < diffThresh * maxIter || pixelsX * pixelsY < pixelSizeThresh) {
    compute(onGPU);                              // leaf: render the pixels
    if (vizMode) ownerFrame->addVizRect({imageX, imageY, pixelsX, pixelsY,
                                depth, true, onGPU});
    ownerFrame->regionComplete();
} else {
    if (vizMode) ownerFrame->addVizRect({imageX, imageY, pixelsX, pixelsY,
                                depth, false, onGPU});
    int subimageX = pixelsX / 2, subimageY = pixelsY / 2;   // upper-left quadrant
     ↪ size
    // ... compute the four sub-rectangles' complex + pixel coordinates ...
    MandelRegion *sub[4];
    sub[UPPER_LEFT]  = new MandelRegion(upperX, upperY, midDiagX1,
     ↪ midDiagY1,
                                imageX, imageY, subimageX, subimageY,
                                ownerFrame, depth + 1);
    sub[UPPER_LEFT]->cornersIter[UPPER_LEFT] = cornersIter[UPPER_LEFT];
     ↪ // inherit corner
    // ... sub[UPPER_RIGHT], sub[LOWER_LEFT], sub[LOWER_RIGHT] likewise
     ↪ ...
    for (int i = 0; i < 4; i++) q.append(sub[i]);
```

```
    ownerFrame->regionSplit();                  // net +3 pending regions
}
```

Two design notes a reader should not miss. First, each child is constructed with `depth + 1` and inherits exactly the one parent corner it shares, so corner evaluation is never duplicated across the tree. Second, the decision rule `diff_uniform OR below_pixT` is why no `diffThreshold` can subdivide the all-interior outlier: its corners (and all nine samples) sit at `maxIter`, so `maxIter - minIter == 0` and the uniformity test passes at every threshold (reports 06, 08).

4.3. compute: pixels, two ways, with metrics

`compute` fills the region's pixels into the frame's `QImage`. The branch on `onGPU` selects the GPU front-end or the CPU loop; both accumulate the same work metrics (mean iterations, interior-pixel fraction) in the same pass, reusing the existing `color == MAXGRAY` branch so the metric costs one add per pixel.

Código 12: mandelregion.cpp — the CPU compute loop (NVTX + timing elided).

```
for (int i = 0; i < pixelsX; i++)
  for (int j = 0; j < pixelsY; j++) {
    double tempx = upperX + i * stepX;
    double tempy = upperY - j * stepY;
    int color = diverge(tempx, tempy);
    iterSum += color;
    if (color == MAXGRAY) {          // MAXGRAY == frame MAXITER == "in
      ↪ the set"
        insetCount++;
        img->setPixel(imageX + i, imageY + j, qRgb(0, 0, 0));   // interior: black
    } else
        img->setPixel(imageX + i, imageY + j, colormap[color]); // exterior: colormap
  }
```

The GPU branch is structurally the same copy-out loop, but the iteration counts come from `hostFE` ($\mathcal{S}6$) rather than from a local `diverge`, and it indexes the pitched result buffer (`h_res[j * pitch + i]`). The CPU branch is wrapped in `clock_gettime` and an NVTX range; both branches emit a per-region metrics line unless `quiet`. The exact line formats are catalogued in `INSTRUMENTATION.md`.

4.4. The interior certificate and the outlier dump

Two diagnostics live in `mandelregion.cpp`. The first is an exact algebraic test for the two largest interior components of the set:

Código 13: mandelregion.cpp — main-cardioid / period-2-bulb membership.

```
bool MandelRegion::inMainCardioidOrBulb(double x, double y) {
  double xm = x - 0.25;
  double q  = xm * xm + y * y;
  if (q * (q + xm) <= 0.25 * y * y) return true;   // main cardioid
  double xp = x + 1.0;
```

```
  if (xp * xp + y * y <= 0.0625)    return true;    // period-2 bulb
  return false;
}
```

A point passing this is provably in the set, so a region whose corners all pass is a candidate for a constant-fill shortcut. The second diagnostic uses it: any CPU region exceeding `OUTLIER_MS` (5000 ms) is dumped in full — regardless of `quiet` — including its corner spread, interior fraction, and `cardioidBulb` flag. That dump is what pinned the binding outliers (frames 73 and 89) and proved they are minibrot interiors with `cardioidBulb=0` — so the cheap cardioid certificate cannot relieve them (report 08). The dump format is in `INSTRUMENTATION.md`, $\mathcal{S}5$.

4.5. The comparator (and a legacy footgun)

`WorkQueue` orders regions by pixel count using a nested functor, made `const` because `std::multiset` invokes its comparator on `const` objects:

Código 14: mandelregion.h — the ordering used by the work queue.

```
struct Compare {
  bool operator()(const MandelRegion *a, const MandelRegion *b) const {
    return a->pixelsX * a->pixelsY < b->pixelsX * b->pixelsY;   // ascending by pixels
  }
};
```

Ascending order means `queue.begin()` is the smallest region and `–queue.end()` the largest — the two ends the queue hands to CPU and GPU workers respectively ($\mathcal{S}5$). There is also a stray member `operator<` on `MandelRegion` left from the original code; it is not what the queue uses (the queue uses `Compare`) and is retained only for historical compatibility. When extending the code, order regions through `Compare`, not `operator<`.

5. workqueue — the Dispatch Policy

The work queue is small but it is where the heterogeneity is exploited. It is a mutex-protected `std::multiset` (not `set`: many regions share a pixel count) ordered by `MandelRegion::Compare`.

Código 15: workqueue.h — the min-max queue and the CPU-only guard flag.

```
class WorkQueue {
  QMutex l;
  multiset<MandelRegion*, MandelRegion::Compare> queue;  // ordered by pixel
    ↪ count
  bool gpuPresent = true;          // false => CPU threads use LPT (see below)
public:
  void append(MandelRegion*);
  MandelRegion* extract(bool isGPU);
  void setGpuPresent(bool g) { gpuPresent = g; }
  int size();
```

```
};
```

Executor-aware extraction — "GPU affinity".

This is the -3.6 % win of report 12. `extract` takes the caller's executor type and hands the GPU thread the largest pending region and CPU threads the smallest:

Código 16: workqueue.cpp — GPU affinity, with the CPU-only LPT guard.

```
MandelRegion* WorkQueue::extract(bool isGPU) {
  QMutexLocker ml(&l);
  if (queue.empty()) return NULL;
  multiset<MandelRegion*, MandelRegion::Compare>::iterator it;
  if (isGPU || !gpuPresent) {       // GPU thread, OR no-GPU run (LPT guard):
     it = queue.end(); --it;       //   take the LARGEST region
  } else
     it = queue.begin();           // CPU thread (GPU present): take the SMALLEST
  MandelRegion* temp = *it;
  queue.erase(it);
  return temp;
}
```

Why this is the lever that moved the wall.

The GPU is ∼30× faster than a CPU thread on a big coherent interior region but only modestly faster on small divergent ones; on the 960 × 540 outlier it is 460 ms versus 13.3 s (report 12). Reserving the largest regions for the GPU therefore moves the expensive coherent work onto the processor that eats it cheaply, while the CPU pool chews through the many small boundary leaves. A shared largest-first queue cannot do this — with eleven CPU threads to one GPU thread, a CPU thread wins the biggest region ∼11:1 (report 11's negative result); the fix is to make extraction depend on who is asking, which is exactly the `isGPU` branch above.

The CPU-only guard.

With `gpuEnable=0`, thread 0 is just another CPU worker; if all threads took the smallest, the big regions would be deferred to the tail (worst-case for makespan). The `!gpuPresent` clause flips CPU threads to largest-first (longest-processing-time, LPT) so big regions start early. `main` wires this once via `workQ.setGpuPresent(enableGPU)`. Measured a wash at this workload's fine granularity, but kept as the principled choice (report 12, $\mathcal{S}$4.4). The graceful-degradation property: when only one size class remains, `begin()` and `–end()` coincide, so no executor idles while work exists.

6. kernel.cu / kernel.h — the GPU Path

The GPU side is three `extern "C"` entry points (so the C++ side links against them by unmangled name) plus the device kernel. Memory is allocated once and reused across every region the GPU thread processes.

One-time setup.

`CUDAmemSetup` allocates pitched device memory (rows padded to an aligned stride for coalesced access) and pinned (page-locked) mapped host memory for fast

transfers, and creates four reusable CUDA timing events.

Código 17: kernel.cu — one-time device + pinned-host allocation.

```
extern "C" unsigned int *CUDAmemSetup(int maxResX, int maxResY) {
  CUDA_CHECK_RETURN(cudaMallocPitch((void**)&d_res, &pitch,
                           maxResX * sizeof(unsigned), maxResY));
  CUDA_CHECK_RETURN(cudaHostAlloc(&h_res, maxResY * pitch,
    ↪ cudaHostAllocMapped));
  cudaEventCreate(&evKernelStart);  cudaEventCreate(&evKernelStop);
  cudaEventCreate(&evMemcpyStart);  cudaEventCreate(&evMemcpyStop);
  return h_res;
}
```

The kernel.

One GPU thread per pixel, in 16×16 blocks (256 threads). Each thread maps its 2D index to a complex coordinate and runs the same escape-time loop as the CPU.

Código 18: kernel.cu — one thread per pixel.

```
___global___ void mandelKernel(unsigned *d_res, double upperX, double upperY,
                      double stepX, double stepY, int resX, int resY,
                      int pitch, int MAXITER) {
  int myX = blockIdx.x * blockDim.x + threadIdx.x;
  int myY = blockIdx.y * blockDim.y + threadIdx.y;
  if (myX >= resX || myY >= resY) return;        // guard the ragged edge
  double tempx = upperX + myX * stepX;
  double tempy = upperY - myY * stepY;
  d_res[myY * pitch / sizeof(int) + myX] = diverge(tempx, tempy, MAXITER);
}
```

A C++ reader should note why interior regions are the GPU's strength here. A warp (32 threads) executes in lockstep; when neighbouring pixels take different iteration counts (the fractal boundary), the warp must wait for its slowest lane — branch divergence. A coherent interior region has every lane running to `MAXITER` together, so there is no divergence and the GPU runs flat-out. This is compounded by the GTX 1660 Ti being consumer Turing, where `double` throughput is 1:32 of single precision; `double` is required for deep-zoom precision, so the GPU's edge comes from coherence, not raw FP64 peak — precisely the regions GPU affinity routes to it.

The host front-end.

`hostFE` launches the kernel, copies the result back, and times both halves with CUDA events. The copy is synchronous, so `cudaMemcpy` implicitly waits for the kernel — which is why its measured time is ∼99 % kernel-wait, not transfer (report 01).

Código 19: kernel.cu — launch, timed memcpy, and event resolution (excerpt).

```
nvtxRangePush("GPU region compute");
dim3 block(BLOCK_SIDE, BLOCK_SIDE);                    // 16 x 16
dim3 grid(ceil(resX/16.0), ceil(resY/16.0));
cudaEventRecord(evKernelStart);
```

```
mandelKernel<<<grid, block>>>(d_res, upperX, upperY, stepX, stepY,
                        resX, resY, ptc, MAXITER);
cudaEventRecord(evKernelStop);
cudaEventRecord(evMemcpyStart);
cudaMemcpy(h_res, d_res, resY * ptc, cudaMemcpyDeviceToHost);  // synchronous
cudaEventRecord(evMemcpyStop);
cudaEventSynchronize(evMemcpyStop);                  // block until stamped
cudaEventElapsedTime(&kernelMs, evKernelStart, evKernelStop);
cudaEventElapsedTime(&memcpyMs, evMemcpyStart, evMemcpyStop);
*pixels = h_res;  *currpitch = ptc;
nvtxRangePop();
```

Because all device/host buffers are allocated once in `CUDAmemSetup` and the events are reused, `hostFE` has no per-call allocation cost — only the launch, the wait, and the copy. `CUDAmemCleanup` prints the aggregate GPU summary and frees everything.

7. The Makefile and the Toolchain

The build is five object files linked by `nvcc`. Two pins are worth understanding:

Código 20: Makefile — the toolchain pins (excerpt).

```
HOSTCC = g++-15
NVCC   = nvcc -ccbin $(HOSTCC)        # nvcc 13.2 does not support gcc 16 yet
CUDA_ARCH = -arch=native              # auto-detect this machine's GPU (CUDA
    ↪ 11.6+)
QT_CFLAGS = $(shell pkg-config --cflags Qt5Core Qt5Gui)   # distro-portable
    ↪ paths
CC_LINK_FLAGS = -lm -lstdc++ $(QT_LIBS) -lpthread -ldl    # -ldl: NVTX3
    ↪ dlopen()
```

First, both halves of the program compile with `g++-15`: `nvcc` uses it via `-ccbin`, and the host `.cpp` files use it directly, so the ABI is consistent across the `nvcc` and `g++` translation units (this is why the `extern "C"` flag symbols in $\mathcal{S}2$ resolve correctly). Second, `-ldl` (not `-lnvToolsExt`) is all NVTX3 needs: the header-only NVTX3 `dlopen()`s the profiler's runtime only when Nsight is attached, so the annotations cost nothing in a standalone run. Qt paths come from `pkg-config` so the Makefile is portable across Arch/Ubuntu. For an older `nvcc`, change `-arch=native` to a concrete arch (e.g. `-arch=sm_61`).

8. The Instrumentation Layer

Every file is wired for profiling; the mechanisms are documented exhaustively (line formats, how to read them, how to run Nsight) in `INSTRUMENTATION.md`. In brief: NVTX ranges (`examine region`, `CPU region compute`, `GPU region compute`) mark each thread's lane in the Nsight Systems timeline; four reused `cudaEvent_t` handles time the kernel and the D→H copy; `thread_local` counters accumulate per-thread CPU time; the `[OUTLIER]` dump fires above 5 s; and one `[total_elapsed_s]` line plus a `[config]` banner make every run self-describing and machine-parseable. All

of it is gated on `quiet`/`viz` and is performance-neutral when off (report 08 verified decomposition-identical output).

8.1. The visualization modes

The three `viz` modes replay the recorded `VizRect`s after the timer stops, stroking each region's outline 2 px inward (so a GPU/CPU boundary shows two solid colour bands instead of two 1 px lines that blend to green at scale). `viz=1` animates one frozen frame depth-by-depth (Figure 2 is one such frame); `viz=2` overlays every frame's full partition; `viz=3` animates the splitting process across the whole run. Modes 2 and 3 force `save=0`. Assemble with `ffmpeg -bf 0 -fps_mode passthrough` (no temporal interpolation, so the lines stay aligned to their frame). The shared overlay routine is compact:

Código 21: main.cpp — inward 2px stroking, coloured by executor (excerpt).

```
auto strokeInward = [&](const VizRect &r) {
   p.drawRect(r.x, r.y, r.w - 1, r.h - 1);          // boundary
   if (r.w > 3 && r.h > 3)
      p.drawRect(r.x + 1, r.y + 1, r.w - 3, r.h - 3); // inset 1px = 2px band
};
// grey skeleton first (split nodes), then leaves: cyan if onGPU else yellow
```

9. How the Code Produces the Measured Behaviour

The design choices above explain the headline numbers from the report series (100 frames, 1920 × 1080, i7-9750H + GTX 1660 Ti Max-Q, 12 threads, `diffT` = 0.1):

- Why the wall is ∼49.9 s and not lower. Adaptive subdivision plus the dynamic queue balance the CPU threads to within <1.4 % of each other across regimes (report 13), so dynamic balance is not the binding constraint. The binding resource is GPU saturation: with affinity the GPU runs ∼91 % busy (report 12), so the ∼49.5 s CPU pool is the wall. Both lanes are full; only reducing work can go lower.
- Why moving work helped but rebalancing did not. Total compute (∼595 s) is conserved by any partitioning — splitting redistributes iterations, it removes none (report 09). So finer subdivision (the 9-point stencil) and shared-queue reordering (report 11) were wall-neutral. GPU affinity moved the wall (−3.6%, 51.73 → 49.86 s, disjoint A/B ranges) because it does something different: heterogeneous assignment, sending the coherent interior regions to the 30×-faster processor.
- Why one region used to dominate. The 960×540 depth-1 outlier is an all-interior minibrot rectangle: its nine samples all hit `maxIter`, spread 0, so `examine`'s uniformity rule computes it whole at every threshold. On a CPU thread that is 13.3 s; routed to the GPU by affinity it is 460 ms, and CPU outliers drop from 1 to 0 (report 12).

In other words: `examine` decides task size (tracking complexity), the `WorkQueue` decides task owner (tracking the processor each task suits), and `MandelFrame`'s atomic counter decides when a frame is done — and those three local decisions compose into the observed global balance.

10. Reading Guide — Where to Go Next

- **To run or profile the binary:** `INSTRUMENTATION.md` (every flag, every stderr line, Nsight recipes, measurement hygiene).
- **For the heuristic's evolution:** reports 03 (the reduction bug), 06 (the `diffThreshold` sweep → 0.1), 08 (region metrics + the outlier diagnosis), 09 (4-corner → 9-point).
- **For the dispatch policy:** reports 11 (shared priority queue — negative), 12 (GPU affinity — the win), 13 (four-regime characterization).
- **To extend the code:** the remaining lever is work-reduction (CLAUDE.md, "Recommended next steps"): periodicity checking inside `diverge()` to bail early on in-set orbits (exact, no image change), or an interior certificate that constant-fills a provably-in-set region. Scheduling has been taken to its limit; the GPU is near-saturated.

11. Conclusion

`mandelHybrid` is a compact study in heterogeneous load balancing: a single recursive heuristic (`MandelRegion::examine`) makes task size track visual complexity, an executor-aware queue (`WorkQueue::extract`) makes task ownership track processor suitability, and an atomic per-frame counter (`MandelFrame::remainingRegions`) makes completion self-detecting — with a CUDA front-end whose strength on coherent interior regions is exactly what the queue feeds it. The five files are small and the control flow is one drain loop shared by every worker, yet the composition reproduces the report series' measured behaviour: CPU threads balanced to $<1.4\,\%$, a conserved $\sim$595 s of compute, the interior outlier offloaded from 13.3 s to 460 ms, and a $\sim$49.9 s wall bounded by GPU saturation. This guide and `INSTRUMENTATION.md` together document the binary as it stands at `binary-v5-affinity`; the open frontier is reducing the conserved compute, not rescheduling it.